Midterm

考试时间：周二 10:00 –13:00, 2.5 h

考试地点：E1003

题型：

1. Multiple choice 20 pts
2. Essay – Q&A
3. Code, Fill the blanks

Ch 1. Enterprise Computing

1. What is enterprise computing?

- **Enterprise**: An organization (company, charity, government department/ Agency)
- **Computing:** The use of computers to process data or perform calculations

What is enterprise computing?

- Enterprise computing is about combing separate applications, services, and process into unified system that is greater than the sum of its parts.
- Enterprise computing is usually seen as a collection of big business IT solutions to common problems such as resource management and streamlining processes.
- People study the common problems (business values) and provide IT solutions (product, platform, framework, service, etc.)
- Enterprise computing involves many different roles (manager, architect, administrator, developer, analyst, operator, support, etc.) for different areas (solution, system, database, network, data, quality assurance, etc.).

- big business IT solution： 面向大型企业、解决复杂业务问题的企业级 IT 解决方案

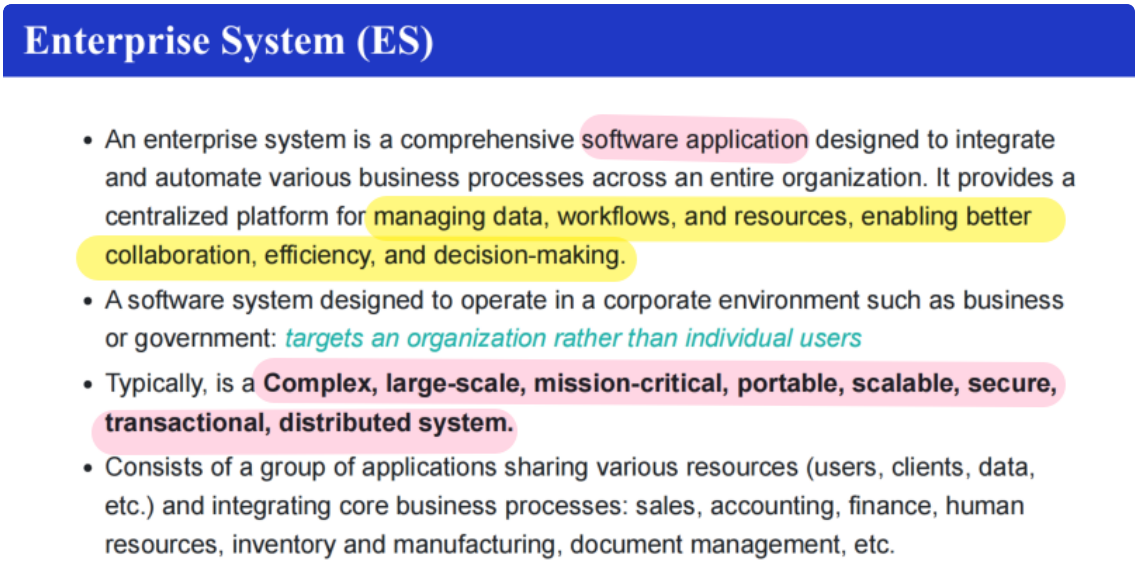
【记忆】

是什么：系统整合 – 把分散的应用、服务和流程整合成统一系统。
为什么：解决共性问题 – 解决资源管理、流程优化等企业常见问题。
怎么做：业务转化为 IT 方案 – 先研究业务问题和价值，再做成产品、平台、框架或服务。
谁来做：多角色跨领域协作 – 管理者、架构师、开发者等人员共同负责系统、数据库、网络等领域。

2. Which enterprise needs enterprise computing?

Retail 零售, Manufacturing 制造, Service, Wholesale 批发, Government, Educational, Transportation

3. Enterprise System (ES)



1. **What ?** –– A comprehensive software application, designed to
   - 整合、自动化业务流程
   - 管理数据、工作流和资源
   - 提高协作、效率和决策能力
2. **Target user?** –– Corporate environment – organization
3. **Character:**

规模与结构	complex、large-scale、distributed
业务性质	transactional、mission-critical
适应能力	portable、scalable
保障要求	secure

4. 组成：
   - **共享资源**：用户、客户、数据等
   - **整合流程**：销售、财务、人力、库存、生产、文档等

【补充 Enterprise system】

企业系统是一类 **整合多个业务流程、跨部门协作的大型软件系统。**

a. Popular enterprise systems

- Human Capital Management (HCM)
- Business Intelligence (BI)
- Content Management System (CMS)
- Customer Relationship Management (CRM)
- Enterprise Application Integration (EAI)
- Enterprise Forms Automation
- Supply Chain Management (SCM)
- Enterprise Resource Planning (ERP)
- Key Performance Indicator (KPI)
- Online Shopping and payment processing.

- **Enterprise Resource Planning (ERP):**
  - Integrates core business processes such as finance, inventory, manufacturing, and sales.
  - SAP, Oracle, Microsoft Dynamics.
- **Customer Relationship Management (CRM):**
  - Manages interactions with customers and prospects, including sales, marketing, and customer service.
  - Salesforce, Microsoft Dynamics, HubSpot.
- **Supply Chain Management (SCM):**
  - Optimizes the flow of goods and services from suppliers to customers, including procurement, production, and logistics.
  - Oracle, SAP, JDA Software
- **Human Capital Management (HCM):**
  - Manages all aspects of the employee lifecycle, from recruitment and onboarding to performance management and payroll.
  - Workday, Oracle, SAP

b. Evolution of Enterprise System

1. Mainframe Era（1960s—1980s）主机时代

Batch processing, centralized computing, and limited user interaction

集中式计算：所有计算都在大型主机上完成

批处理（Batch Processing）：每天或每周跑批任务

用户交互极少：通常是命令行或卡片输入

2. Client—Server Era（1980s—2000s）客户端–服务器时代

Distributed computing, GUIs, networking… relational databases.

分布式计算：客户端负责 UI，服务器负责逻辑

图形界面（GUI）：用户体验大幅提升

网络化：企业开始内部联网

3. ERP Era（1990s—Present）ERP 时代

Integrated business processes… real–time information… enterprise–wide integration

业务流程整合：财务、库存、制造、销售全部统一

实时数据：企业第一次能实时看到业务状态

标准化流程：企业流程被软件固化

4. Web–Based Enterprise Systems（2000s—Present）基于 Web 的企业系统

Web accessibility… cloud computing… reduced infrastructure costs.

浏览器访问：不再需要安装客户端

SaaS 模式出现，部署成本下降

5. Mobile & Cloud-Native Era（2010s—Present）移动与云原生时代

Mobile accessibility… cloud-based infrastructure… microservices architecture.

移动化：随时随地访问企业系统

云原生：系统为云而生，而不是迁移到云

微服务架构：系统拆成小服务，独立部署

Mainframe Era 1960s—1980s	“Focus: Batch processing, centralized computing, and limited user interaction.”	**集中式**计算、批处理、低交互、昂贵且难维护	IBM Mainframe、COBOL、早期库存/财务系统
Client—Server Era 1980s—2000s	“Distributed computing, GUIs, networking… relational databases.”	分布式计算、图形界面、网络化、关系型数据库	Oracle、SQL Server、SAP R/2、桌面应用
ERP Era 1990s—至今	“Integrated business processes… real-time information… enterprise-wide integration.”	业务流程整合、实时数据、企业级**统一平台**	SAP R/3、Oracle EBS、Microsoft Dynamics
Web-Based ES 2000s—至今	“Web accessibility… cloud computing… reduced infrastructure costs.”	Web 化访问、SaaS 出现、部署成本下降	Salesforce、Workday、NetSuite
Mobile & Cloud-Native Era 2010s —至今	“Mobile accessibility… cloud-based infrastructure… microservices architecture.”	移动化、云原生、微服务、高扩展性	AWS/Azure/GCP、Kubernetes、Docker、SAP S/4HANA

c. Benefits of enterprise system

- Improved efficiency and productivity 提效
- Enhanced data visibility and reporting 可视化
- Increased collaboration and communication 合作
- Better decision-making based on real-time data 决策
- Reduced costs through automation and process optimization 花销
- Improved customer service and satisfaction 用户满意
- Enhanced compliance with regulations 合规

d. Challenges

- High implementation costs
- Complexity and integration challenges
- Resistance to change from employees
- Potential for data security breaches
- Ongoing maintenance and support requirements

在自动化和流程优化的 cost 减少了，但是实现过程中的 cost 反而增加

Ch1. Enterprise applications

1. Enterprise applications features.

Enterprise system Key characters

1. High Performance

- Able to provide rapid transaction time even in peak hours.
- Able to process massive amount of transactions concurrently 并发处理

2. Stable & Robustness 稳定性与健壮性

- Server running consistently and correctly under any kinds of legal and illegal operations.
- Constantly amend and improve to prevent loopholes. 持续修补和改进以防止漏洞
- System environmental upgrade to fix potential problems.

3. Resilience Control 弹性控制 / 韧性设计

- Anti-virus, malicious software prevention. 防病毒、恶意软件防护
- Power supply shortage. 应对电力供应不足等突发情况。

4. Cost Effectiveness 成本效益

Short development effort but high profitable product.　开发周期短，但产品利润高。

5. **Extensible & Flexible** 可扩展与灵活性

- Ease of further development and integration with others. 能轻松增加新模块、新功能；能与第三方系统集成

6. **Disaster Recovery (DR) 灾难恢复**

- Data & software backup and recovery mechanism.
- Data Centers and DR sites are required.

DR 是企业系统面对“灾难级事件”时的 **恢复能力**，属于系统可靠性最高层级。

它解决的问题是：数据丢，机房损坏，火灾、洪水、地震，整个区域的服务不可用

7. **High Availability (HA) 高可用性**　HA 的目标是：**系统永不停机**。

- Provide almost zero downtime services (24 × 7).
- Load balancing, failover, virtual servers, etc. 负载均衡、故障切换、虚拟服务器

8. **Resource Management & Optimization**

Database, application servers, operating system, network optimization. 企业系统规模大，资源消耗高，因此必须进行系统级优化

9. **Standardization & Documentation 标准化与文档化**

- Provide **enough documents** (system handbooks, operational guidelines, administration whitepaper, etc.) to explain the system.
- Acquire ISO to reduce operational risks, maintain high quality of services and secure to achieve industrial standard level. 获得 ISO 等行业标准，以降低运营风险、保持高质量服务并达到行业标准
- Code of Practice for **staffs** to follow the operational procedures.

Ch2. Application Architectures

1. What is an Architecture Pattern?

Enterprise architectural styles are high-level patterns and principles that guide the design and structure of an enterprise's IT systems and infrastructure. These styles help ensure that the systems are scalable, maintainable, and aligned with the organization's business goals.

- Layered Architecture
- Microservices Architecture
- Service-Oriented Architecture (SOA)
- Event-Driven Architecture (EDA)
- Client-Server Architecture
- Space-Based Architecture

企业架构风格是高层次的**模式和原则**：

- 指导：IT 系统的**设计、结构、基础设施**
- 保证：系统**可扩展、可维护、符合业务目标**

6 个架构：

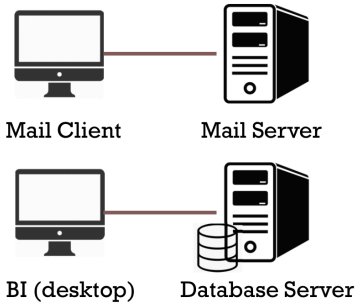
- **层**：Layered Architecture，分层架构
- **微**：Microservices Architecture，微服务架构
- **服**：Service-Oriented Architecture，面向服务架构
- **事**：Event-Driven Architecture，事件驱动架构
- **客**：Client-Server Architecture，客户端—服务器架构
- **空**：Space-Based Architecture，空间架构

2. Multi-tiler Architectures 多层部署架构

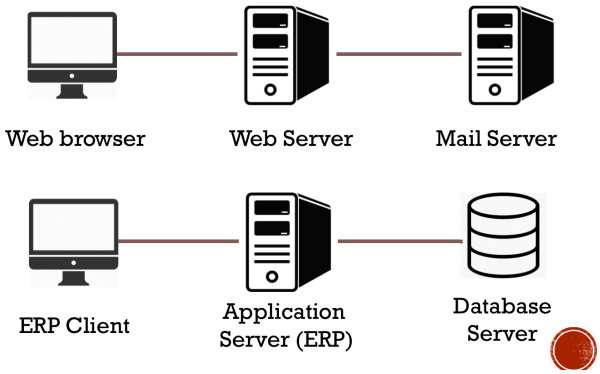
核心问题： 一项请求要经过几个独立的客户端或服务器

MULTI-TIER ARCHITECTURES

- 2-tier:
  - 1 client and 1 server
  - Popular in simple LANs



- 3-tier:
  - 1 client and 2 servers

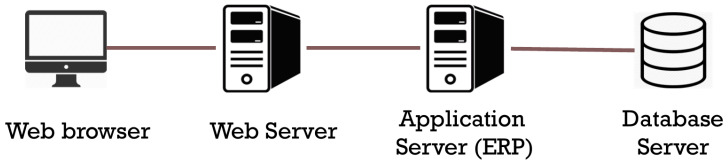


1. **两层架构**：客户端直接找服务器 | Client → Server
2. **三层架构**：中间增加业务服务器 | 表示层 → 业务逻辑层 → 数据层
   - **客户端**：显示界面、接收用户操作
   - **应用服务器**：执行订单计算、权限验证等业务逻辑
   - **数据库服务器**：保存和查询数据

MULTI-TIER ARCHITECTURES

The application logic is distributed across 2 or more servers

- n-tier:
  - more then 3 elements



- **Pros**
  - **Distributed workloads**
  - **Easy to scale**
  - **Flexibility**
- **Cons**
  - **Complexity**
  - **Hard to troubleshoot**

N 层＝继续拆分，各司其职

- 浏览器：显示页面
- Web 服务器：接收 HTTP 请求、提供网页
- 应用服务器：处理 ERP 业务逻辑
- 数据库服务器：存储数据

为什么要拆成更多层?

因为不同部分的压力不同，可以分别扩展。例如：

- 网页访问量大：增加 Web 服务器
- 业务计算量大：增加应用服务器
- 数据查询量大：增强数据库或增加副本

优点：

- **Distributed workloads**：分散工作负载
- **Easy to scale**：容易扩展
- **Flexibility**：灵活，各层可以独立修改

缺点：

- **Complexity**：系统更加复杂
- **Hard to troubleshoot**：请求经过的环节多，故障难排查

3. Benefits of layering

- Layering is one of the most common techniques that software designers use to break apart a complicated software system.
- Breaking down a system into layers has a number of important benefits
  - You can understand a single layer as a coherent whole without knowing much about the other layers.
  - You can substitute layers with alternative implementations of the same basic services.
  - Layers make good places for standardization
  - Once you have a layer built, you can use it for many higher-level services.
  - You minimize dependencies between layers.

1. 独立理解：不需要理解其他层
2. 方便替换：只要提供相同的基本服务，就能替换某一层
3. 便于标准化：每层都适合指定统一标准
4. 可以复用：建好一层后，可以被多个上层服务使用
5. 减少依赖：尽量减少各层之间的依赖

Ch3. Java EE Application Architecture

1. What is java EE ?

- **What is?** The aim of the Java EE platform is to provide developers with a powerful set of API while

shortening development time, reducing application complexity, and improving application performance.

- **Specifications**: Java EE API describes how an enterprise application should be created, what components should contain, etc
- **Implementations** (Application Servers) : GlassFish, Payara,WeblogicServer, Jboss etc

2. Advantages of using Java in Enterprise

- **Java can run on multi-platform (WORA)**
- **Object Oriented design for system architecture**
- **Plenty of framework for system building**
- **Wide dissemination with minimal cost**
- **Dominating the e-Commerce industry**
- **FOSS (free and open source software)**

1. **Java can run on multi-platform (WORA)**

Java 可以在多个操作系统上运行。WORA 是：Write Once, Run Anywhere， 即“编写一次，到处运行”

Java 源代码不会直接编译成特定操作系统的机器码，而是先编译成字节码：

只要操作系统安装了相应的 JVM，同一份 `.class` 文件通常不需要重新编译即可运行

```
Java source code (.java)
          ↓ javac
Java bytecode (.class)
          ↓ JVM
Windows / Linux / macOS
```

2. **Object Oriented design for system architecture** Java 是一种面向对象语言，适合使用类和对象组织大型系统

例如，一个电商系统可以分成：Customer, Product, ShoppingCart, Order...

每个类负责一部分**数据和行为**。这种方式有助于：

- 将大型系统拆成较小模块
- 降低不同模块之间的依赖

3. **Plenty of framework for system building** Java 拥有大量成熟框架和工具，例如：

- Spring：依赖注入和企业应用开发
- Spring Boot：简化应用配置和部署

- Hibernate：数据库对象关系映射
- Jakarta EE：企业级 API 规范
- Maven、Gradle：依赖和项目构建管理
- JUnit：自动化测试

这些框架解决了许多重复问题，使开发者可以把更多精力放在业务逻辑上。

4. **Wide dissemination with minimal cost** Java 传播范围广，而且使用成本相对较低。

原因包括：

- Java 开发工具容易获得
- OpenJDK 可以免费使用
- 有大量免费的框架和程序库
- 能在普通服务器和主流操作系统上运行
- 社区规模大，学习资料丰富

对于企业而言，可以使用开源 Java 生态建立系统，而不一定要购买昂贵的专有开发工具

5. **Dominating the e-Commerce industry** 在电子商务和大型企业后端系统中曾占有重要地位。

Java 的并发支持、企业框架、事务管理、安全机制和成熟生态适合这些需求。因此，许多大型电商、银行和企业后台长期使用 Java。

6. **FOSS (free and open source software)** 即“自由及开源软件”。

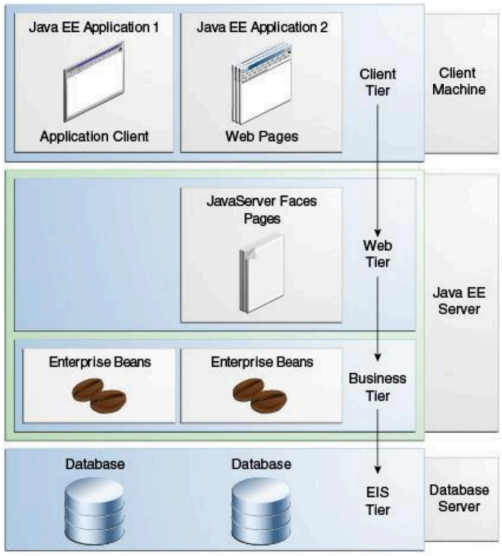
- **Free** 更强调使用、研究、修改和分发软件的自由，不一定只表示价格免费。
- **Open Source** 表示软件源代码开放，并允许按照许可证使用和修改。

3. Java EE Application Architecture

In a class web-based Java EE application architecture, there are four tiers:

1. Client Tier
2. Web Tier
3. Business Logic Tier
4. Enterprise Information System (EIS) Tier.

- Java EE applications are made up of ***components***



a. 第一层：Client Tier

Client Tier 是用户直接接触的**前端**：它负责：接收用户输入 -> 向服务器发送请求 -> 显示服务器返回的结果

常见组件包括：Web pages, application client, 早期的 Java applet

b. 第二层：Web Tier

- **Web Tier**
  - **Java servlet**: Java servlets are server side modules, typically used to process a request and send back response in the web applications.
  - **Java Server Pages (JSP)**: JSPs are also server side modules used to process the web requests. JSPs (Java Server Pages) are great for handling requests that generate large HTML markup responses.
  - **Java Server Faces (JSF)**: JSF makes creating user interface on the server side modular by incorporating the MVC design pattern in its implementation.

Web Tier 接收客户端请求，**调用业务逻辑，并生成响应**。

c. 第三层：Business Logic Tier

- **Business Logic Tier**
  - **Enterprise Java Beans (EJB)**: are the Java classes where you can write your business logic.

Business Logic Tier 负责真正的业务规则。

d. 第四层：Enterprise Information System Tier

它表示企业应用所连接的后端信息系统

- **Enterprise Information System (EIS) Tier**
  - **Java Database Connectivity (JDBC)**: JDBC is a specification to access relational database in a common and consistent way.
  - **The Java Persistent API (JPA)**: JPA gives you the flexibility to map the objects to the tables in relational database and execute the queries with or without using Structured Query Language (SQL).
  - **Web Services**: Web services are remote application components that expose self-contained APIs.
    - Simple Object Access Protocol(SOAP)
    - Representational State Transfer (REST).

它表示企业应用所连接的后端信息系统

4. Common Multi-tier Applications

- **Client Tier:** Web pages, application clients
- **Web Tier:** Java servlet, JSP, JSF
- **Business Logic Tier**: EJB
- **Enterprise Information System Tier**: JDBC, JPA, Web service

【解释】

1. **JDBC**：它允许 Java 程序直接执行 SQL：

JDBC 接近数据库底层，开发者通常需要自己：编写 SQL，读取查询结果，将结果转换成 Java 对象，处理连接和异常

2. **JPA**：它用于把 Java 对象**映射**为数据库表，这种机制称为 ORM：Object-Relational Mapping}

JDBC 与 JPA 的关系可以简单理解为：

Plain Text

```
JPA：较高层的对象操作方式
 ↓
JDBC：较底层的数据库访问方式
 ↓
关系型数据库
```

3. **Web Services**：Web Service 允许系统通过网络调用另一个系统提供的功能。

Web Services: Web services are remote application components that expose self-contained APIs.

- Simple Object Access Protocol(SOAP)
- Representational State Transfer (REST).

- SOAP：SOAP 通常使用 XML，具有较**严格的消息格式和服务契约**。在传统企业系统、政府系统和银行系统中较常见。

Plain Text

```
系统A → SOAP XML消息 → 系统B
```

- REST，REST 通常使用 HTTP 方法：GET, POST, PUT, DELETE

5. Java EE technologies (JSP, Servlet, JSF, JPA ...) Java EE / Jakarta EE 中常见的服务器端技术

- **Java servlet**: Java servlets are **server** side modules, typically used to process a request and send back response in the web applications. Java Servlet 是服务器端组件，通常负责处理客户端请求，并向客户端返回响应

- **Java Server Pages (JSP)**: JSPs are also server side modules used to process the web requests. JSPs (Java Server Pages) are great for handling requests that generate large HTML markup responses. JSP = JavaServer Pages， JSP 是服务器端页面技术，特别适合生成包含大量 HTML 标记的响应
- **Java Server Faces (JSF)**: JSF makes creating user interface on the server side modular by incorporating the MVC design pattern in its implementation. JSF = JavaServer Faces， JSF 是服务器端用户界面框架，通过采用 MVC 设计模式，使 UI 的创建更加模块化。
- **The Java Persistent API (JPA)**: JPA gives you the flexibility to map the objects to the tables in relational database and execute the queries with or without using Structured Query Language (SQL). JPA = Java Persistence API， JPA 提供一种标准方式，把 Java 对象映射到关系数据库中的表。

Servlet	请求控制器	接收请求、调用逻辑、返回或转发响应
JSP	页面模板	生成 HTML
JSF	UI 组件框架	构建组件化界面，管理事件、验证和导航
JPA	持久化规范	将 Java 对象映射并保存到数据库

Ch3. Fundamental Server Components

1. Web Server, web Container and Application Server

a. Containers

组件本身只包含应用逻辑，但它还需要许多基础功能才能运行

Containers are the interface between components and the low-level platform-specific functionality that supports the component.

- **Before a web, enterprise bean, or application client component can be executed, it must be assembled into a Java EE module and deployed into its container"**
- **Web/servlet Containers** manage the execution of web pages, Java servlets, JSP, JSF etc. Implemented in most application servers, such as Tomcat.
- **Java EE/EJB containers** manages the execution of EJB, JMS, etc. and are implemented in the "heavy" Java EE servers, such as ***Glassfish***.

Component 写业务逻辑，Container 提供运行环境

- Web Container，管理 Web 组件，例如，Servlet，JSP，JSF，WebSocket
- EJB Container，管理 EJB 等企业业务组件。

b. Web server

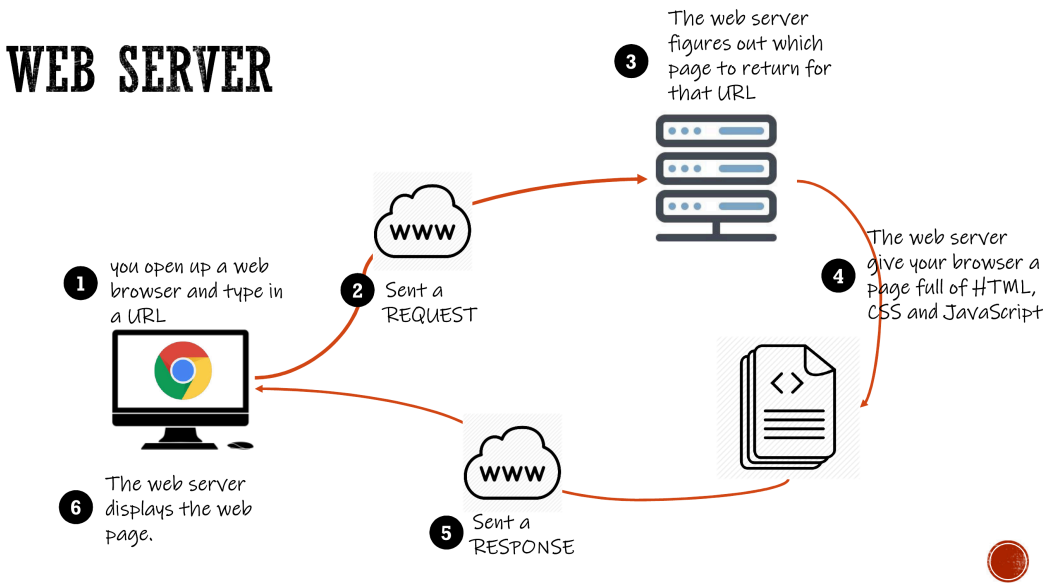
Web server 是通过 HTTP 向客户端提供 Web 内容的软件。

A web server is software that helps to deliver web content (web pages) to the client (e.g. web browser) through the Internet using HTTP protocol.

- Handle static web contents with **HTTP protocol**
  - It processes the HTTP Request and Response to deliver **static web contents** (e.g. web pages, images, documents, etc.).
  - It doesn't process the server-side programs (PHP, .NET, Java, etc.) to deliver dynamic web contents
- Use **port 80** to provide services in general
  - it reserves this port to provide HTTP services as default.
  - Once it is started, you cannot run other services to use the same port.

1. **静态内容**：Web server 最擅长处理静态内容，也就是服务器上已经存在的文件，如 web pages, images, documents，Web server 会读取文件并返回。
2. **动态内容**： **Web server** 本身通常**不负责执行**复杂的**服务器端程序**， **动态页面**需要应用程序**根据**用户、数据库或**请求参数生成**。因此通常需要 Web Server 把**请求转交给后端应用**，或者由 Application Server 处理。

浏览器访问网页的过程



常见 Web Server

February 2021

Below are the latest statistics of the market share of all sites of the top web servers on the Internet by Netcraft February 2021 Web Server Survey

Product	Vendor	Percent
nginx	NGINX, Inc.	34.54%
Apache	Apache	26.32%
IIS	Microsoft	6.5%
OpenResty	OpenResty Software Foundation	6.36%
Cloudflare Server	Cloudflare, Inc.	5.0%

https://en.wikipedia.org/wiki/Web_server#Market_share

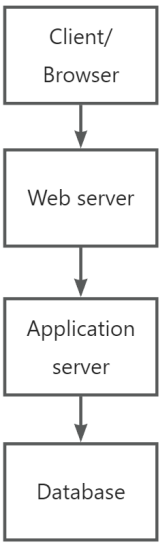


c. Application Server

Application server 是能够托管和运行应用程序的服务器软件。

Java EE Application Server

- An Application server is a server that hosts applications.
- Application servers consist of
  - Web server connectors,
  - Computer programming Languages,
  - Runtime libraries,
  - Database Connectors,
  - Administration code needed to deploy, configure, manage and connect these components on a web host.
- An application server runs behind a web server and (almost always) in front of a database.



Web Server 主要负责：

接收请求 → 返回文件

Web Server 的作用

- 接收外部 HTTP 请求；
- 提供**静态文件**；
- 作为反向代理；
- 过滤和**转发**请求；
- 处理 HTTPS；
- 进行负载均衡。

Application Server 还可以负责：

接收请求
→ 执行业务逻辑
→ 访问数据库
→ 管理事务
→ 处理安全
→ 返回动态结果

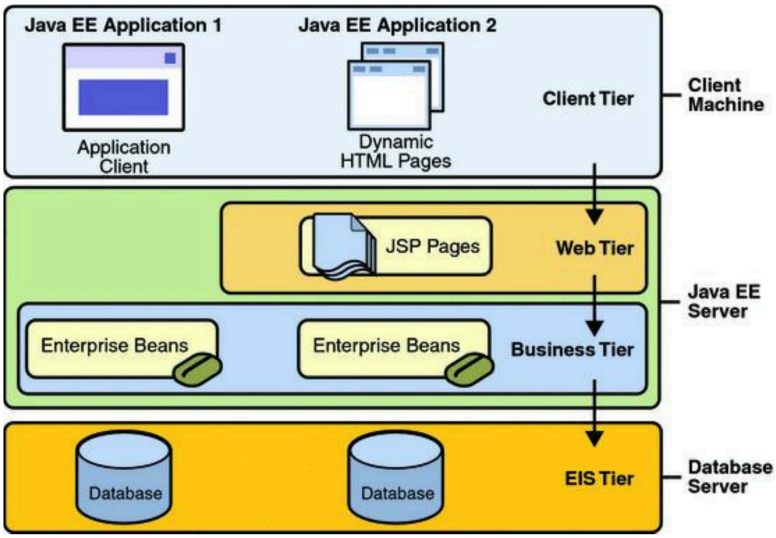
Application Server 的作用

- 执行后端应用；
- 处理业务逻辑；
- 管理事务；
- 管理数据库连接；
- 处理用户权限；
- 运行 Java EE 组件。

Java EE Application Server

Java EE 定义了企业应用服务器应该支持的一组 API 和功能。

- Java EE defines the core set of API and features of Java Application Servers.
- A Java EE application server contains an EJB container as well as a servlet container and implements all the required specifications of Java EE.



Web Container

Web Container 管理：**Servlet**，JSP，JSF，WebSocket

它负责处理 Web 请求、Servlet 生命周期和 Session 等内容

EJB Container

EJB Container 管理 Enterprise JavaBeans，负责：业务逻辑组件，事务，安全，并发，生命周期，依赖注入，远程调用

项目	Web Server	Web Container	Application Server
主要任务	返回静态网页	运行 Web 组件	运行完整企业应用
典型内容	HTML、CSS、图片	Servlet、JSP	Servlet、EJB、JPA、JMS 等
业务逻辑	通常很少	可以调用业务层	可以完整执行
事务管理	通常不负责	有限	支持
数据库连接	通常通过代理	可以连接	提供完整管理
例子	Nginx、Apache、IIS	Tomcat	GlassFish、WildFly、WebLogic

2. Port Number. (Glassfish)

HTTP	8080
HTTPS	8181
Admin Console	4848

GlassFish 是一个 Java EE Application Server 实现。

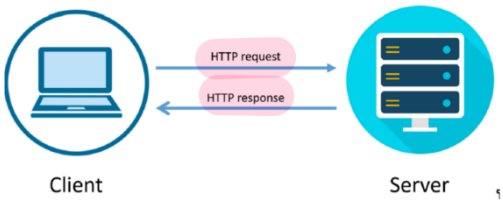
- **Glassfish** is an open-source and commercial full Java EE application server implementation.
- It provides all the features in the Java EE specification, including a web container and an EJB container.
- It will be used throughout this course.



Ch4. Web APP architecture

1. HTTP protocol (Request/Response, status codes)

- Hyper Text Transfer Protocol.
- Communication between web server & clients.
- Runs on top of TCP/IP.
- The structure of an HTTP conversation is a simple Request / Response sequence.



Request:

- Key elements of the request stream:
  - HTTP method (the action to be performed)
  - The page to access (a URL)
  - Form parameters (like arguments to method)

Response:

- Key elements of the response stream:
  - A status code (for whether the request was successful)
  - Content-type (text, picture, HTML, etc.)
  - The content (the actual HTML, image, etc.)

Request

- Method：要做什么
- URL：对谁做
- Parameters：具体要求是什么

```
GET / HTTP/1.1
Host: localhost
```

```
POST /users
Content-Type: application/json

{"name": "Tom"}
```

Response

- Status code：处理得怎么样
- Content-Type：返回的是什么格式
- Content：实际返回的内容

The HTTP response is made up of three parts:

- the status line: `HTTP/1.0 200 OK`
- header
- response body.

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 10, "name": "Tom"}
```

- Status Codes
  - 1xx – informational
  - 2xx – Success!
  - 3xx – Redirection
  - 4xx – Client error
  - 5xx – Server error

类别	含义
1xx	信息：正在处理
2xx	成功
3xx	重定向：去别处

`4xx`	客户端出错
`5xx`	服务器出错

2. Parts of a URL

```
http://localhost:8080/MyApplication/myServlet?name=Tom
|      |         |          |          |
|      |         |          |          └─ Query String / Parameter
|      |         |          └──────────── Resource (Servlet)
|      |         └─────────────────────── Context Path
|      └───────────────────────────────── Host + Port
└──────────────────────────────────────── Protocol

```

Part	Example
Protocol	http
Host	localhost
Port Number	8080
Context Path	MyApplication
Resource Path	myServlet
Query String	name=Tom

Plain Text

```
http://localhost:8080
```

- Host = localhost
- Port = 8080

以及：

Plain Text

```
request.getParameter("name")
```

对应 URL 的：

Plain Text

```
?name=Tom
```

3. HTTP Methods

In any different types of HTTP request they must contain a HTTP method which tell the server what the client is asking it to do.

- GET
  - Retrieves data from the server
- POST
  - Submit data to the server
- PUT
  - Update data already on the server
- DELETE
  - Deletes data from the server
- Other methods
  - Online documents

`GET`	获取数据	Read	doGet()
`POST`	新增或提交数据	Create	doPost()
`PUT`	更新或整体替换数据	Update	doPut()
`DELETE`	删除数据	Delete	doDelete()

4. Basic Java EE Web application structure (DD, WAR)

Deployment Descriptor (DD) which includes instructions for deploying and starting your application

You must **package** your application somehow, and for that you have WAR and EAR files.

a. DD：Deployment Descriptor，部署描述符

在 Java Web 应用中，DD 通常就是： WEB-INF/web.xml

它是一个 XML 配置文件，用来告诉 Web 容器（如 Tomcat）： 这个应用应该如何启动、访问和管理

Deployment Descriptor (web.xml)

- To describe the web application
  - Define default homepage
  - Define Java Servlet mapping path
  - Define an event listener class
  - Set up resource reference
  - Session control
  - Etc.
- Full Documentation
- There are plenty of tools for building the web.xml such as Apache Ant, Maven, eclipse, etc.

Welcome page	默认首页	用户只输入网站地址时显示哪个页面
Servlet mapping	Servlet 路径映射	哪个 URL 交给哪个 Servlet
Event listener	事件监听器	监听应用启动、关闭、会话创建等事件
Resource reference	资源引用	配置数据库、邮件服务等资源
Session control	会话控制	设置 Session 超时时间
Security configuration	安全配置	配置角色、权限和受保护路径

Web.xml Example

- Homepage setting allows user not to type the name of the first page.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1"
         xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd">

    <welcome-file-list>
        <welcome-file>default.htm</welcome-file>
        <welcome-file>index.htm </welcome-file>
        <welcome-file>index.html  </welcome-file>
        <welcome-file>index.jsp  </welcome-file>
    </welcome-file-list>
```

它定义了 welcome files（默认首页），服务器就依次查找：`default.htm`，`index.htm`，`index.html`，`index.jsp`，找到第一个存在的文件，就把它返回。

b. WAR: Web Application Archive，Web 应用归档文件

Is a JAR file to distribute a collection of JSP, Servlet, class, XML, tag library, HTML pages, etc

它是一个用于打包和部署 Java Web 应用的归档文件，可以理解为 一个可以直接交给服务器部署的应用包

它和 JAR 一样，本质上通常采用 ZIP 格式，但内部目录结构必须符合 Java Web 规范

- WAR (Web Application ARchive)
  - Is a JAR file to distribute a collection of JSP, Servlet, class, XML, tag library, HTML pages, etc.
  - Can be digitally signed as JAR file to assert the code is trusted

5. WAR folder structures (/WEB-INF/classes/)

- Organize the programs for web container
- Web Container will unpack the files
- Root folder (WebContent/app.)
  - JSP, HTML, image, JavaScript, CSS
  - META-INF
    - MANIFEST.MF
  - WEB-INF
    - web.xml (Deployment Descriptor)
    - classes
      - Java Servlet, JavaBean, properties
    - lib
      - JAR files, DLL files, other library files
    - tages
      - JSP Tag Library

```
MyApp.war
│
├── index.html
├── jsp files
├── css
├── js
│
├── META-INF
│   └── MANIFEST.MF
│
└── WEB-INF
    ├── web.xml
    ├── classes
    │   ├── Servlet.class
    │   ├── JavaBean.class
    │   └── properties
    │
    └── lib
        ├── xxx.jar
        └── yyy.jar
```

`/WEB-INF/classes/` contains **compiled Java classes**, such as **Servlets** and **JavaBeans**, as well as application resource files such as **properties file.**

`/WEB-INF/classes/` 存放编译后的 Java 类，例如 Servlet 和 JavaBean，以及 properties 等应用配置资源文件

注意实际存进去的是编译后的 `.class` 文件，而不是通常意义上的 `.java` 源代码。

6. 8 steps in the HTTP request and response flow from a servlet

Ch4. Java Servlets (Important)

A Servlet is a **server-side Java component** running in a web container. It receives an HTTP request, executes application logic, and generates an HTTP response. Servlet 是运行在 Web 容器中的 Java 服务器端组件：**接收 HTTP Request**，执行业务逻辑，**生成 HTTP Response**

Servlet 不是独立运行的普通 Java 程序，它运行在 **Web Container（Web 容器）** 中，如 Apache Tomcat, GlassFish...

Web 容器负责：

Create the Servlet object	创建 Servlet 对象
Manage its life cycle	管理 Servlet 生命周期
Receive HTTP requests	接收 HTTP 请求
Map URLs to Servlets	把 URL 映射到 Servlet
Create request and response objects	创建请求与响应对象
Manage threads	管理处理请求的线程
Handle security and sessions	处理安全和会话

因此， 容器管理 Servlet，Servlet 处理请求

1. Java Servlet Interface and HttpServlet class

Servlet 的生命周期由容器管理，包括 `init()` 、 `service()` 和 `destroy()` 。实际开发通常继承 `HttpServlet` ，并重写 `doGet()` 、 `doPost()` 等方法。

The `Servlet` Interface (Java Interface)

```
package javax.servlet;

public interface Servlet {

    public void init(ServletConfig config) throws ServletException;

    public void service(ServletRequest req, ServletResponse res)
        throws ServletException, IOException;

    public void destroy();

    public ServletConfig getServletConfig();

    public String getServletInfo();

}

public abstract class GenericServlet implements Servlet { ... }

public abstract class HttpServlet extends GenericServlet { ... }
```

Servlet 接口的五个方法

- **`init(ServletConfig config)`** ： Initializes the Servlet. 初始化 Servlet，并接收配置对象。
- **`service(req, res)`** ： Processes a client request and produces a response. 处理客户端请求并产生响应。
- **`destroy()`** ：Releases resources before the Servlet is destroyed. Servlet 销毁前释放资源
- **`getServletConfig()`** ：Returns the Servlet configuration object. 返回 Servlet 的配置信息
- **`getServletInfo()`** Returns information about the Servlet, such as its author or version. 返回 Servlet 的作者、版本或描述等信息。

三个类之间的继承关系

1. `Servlet` ：最基础的接口，规定五个方法。**Servlet defines the basic life-cycle contract.**

如果直接实现它，就必须实现全部方法：

Plain Text

```
public class MyServlet implements Servlet {
    // Must implement all five methods
}
```

2. `GenericServlet`

它提供了 Servlet 接口的通用实现，因此开发者不必自己实现所有方法。 Plain Text

```
public abstract class GenericServlet
        implements Servlet
```

它提供了 Servlet 接口的通用实现，因此开发者不必自己实现所有方法。

GenericServlet 不特定于某种网络协议。

3. `HttpServlet`

Plain Text

```
public abstract class HttpServlet
        extends GenericServlet
```

它专门处理 HTTP 请求，并提供：

Plain Text

```
doGet()
doPost()
doPut()
doDelete()
```

- A HTTP servlet is a class that extends the `javax.servlet.HttpServlet` class, which is subclass of `javax.servlet.GenericServlet`.
- A servlet must implement one or more methods to respond to specific HTTP requests. These methods are overridden from the parent `HttpServlet` class.
- GET: `doGet(HttpServletRequest request, HttpServletResponse response)`
- POST: `doPost(HttpServletRequest request, HttpServletResponse response)`
- PUT: `doPut(HttpServletRequest request, HttpServletResponse response)`
- DELET: `doDelete(HttpServletRequest request, HttpServletResponse response)`

- `service()` 负责分发 dispatches the request， `doXxx()` 负责处理。
- HTTP Servlet 是一个继承 `HttpServlet` 的 Java 类
- `HttpServlet` 已经定义了处理各种 HTTP 方法的函数。开发者通过 **override（重写）**，加入自己的业务逻辑辑

类型	作用
`Servlet`	定义 `init()` 、 `service()` 、 `destroy()` 等基本接口
`GenericServlet`	提供与协议无关的通用实现
`HttpServlet`	专门支持 HTTP，提供 `doGet()` 、 `doPost()` 等方法

`UserServlet`	开发者自己编写的业务 Servlet

The `HttpServlet` Class

```
package javax.servlet.http;

public abstract class HttpServlet extends GenericServlet {

    protected void service(HttpServletRequest req, HttpServletResponse resp)
      throws ServletException, IOException {

        String method = req.getMethod();

        if (method.equals(METHOD_GET)) {
            doGet(req, resp);
        } else if (method.equals(METHOD_POST)) {
            doPost(req, resp);
        } ...
}
```

```
/*
  Called by the server (via the service method) to allow a servlet to
  handle a GET request. Override this method to support a GET request.
*/

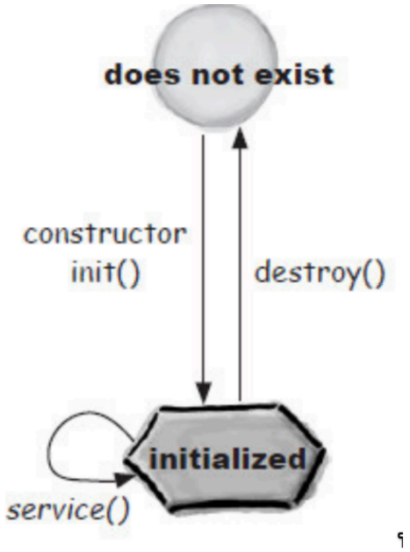
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
  throws ServletException, IOException {
}


protected void doPost(HttpServletRequest req, HttpServletResponse resp)
  throws ServletException, IOException { ... }
...
}
```

2. Java Servlet Lifecycle（p56）

Servlet 生命周期是 Servlet 从创建到销毁的整个过程： 加载 → 创建 → 初始化 → 服务 → 销毁

- A complete Servlet program consists of three methods
  - `init(), service(), destroy()`
- Stay most of the life time on `service()`
  - Require to implement `service()`
  - `doGet(), doPost()`
- Other commonly used API
  - `ServletConfig`
  - `ServletContext`
  - `ServletContextListener`
- Where will the Java Servlet runs on
  - Java Web Container
  - Microsoft IIS **doesn't** support Java



1. **加载 Servlet**： 类加载器（class loader）负责加载 Servlet 类， 当 Web 容器第一次收到该 Servlet 的请求时，Servlet 类会被加载到 Web 容器中（如 Glassfish）
2. **创建 Servlet 实例**： 加载 Servlet 类之后，Web 容器创建 Servlet 实例， 在一次 Servlet 生命周期中，Servlet 实例只创建一次
3. **调用 `init()` 初始化**：Servlet 实例创建成功后，Web 容器调用 `init()` 方法对其进行初始化， `init()` 代表 Servlet 从“已经创建”进入“已经初始化”的状态。这一阶段结束后，Servlet 才能开始处理请求。
4. **调用 `service()` 处理请求**：Servlet 通过 `service()` 方法处理客户端请求， 在生命周期的大部分时间里，Servlet 都处于接收和处理请求的阶段。

因此图中是一个循环：

```
initialized
     ↺ service()
```

表示 Servlet 初始化后可以不断处理客户端请求：

```
第一个请求 → service()
第二个请求 → service()
第三个请求 → service()
```

5. 调用 `destroy()` 销毁：Servlet 通过调用 `destroy()` 方法结束生命周期，调用 `destroy()` 后，Servlet 结束运行，不再处于已初始化状态。

A servlet life cycle can be defined as the entire process from its creation to its destruction. The following are the paths followed by a servlet.

- Servlet class is loaded into web container (Glassfish)
- Servlet instance is created (constructor run)
- The servlet is initialized by calling the `init()` method.
- The servlet calls `service()` method to process a client's request.
- The servlet is terminated by calling the `destroy()` method.

- The *class loader* is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.
- The *web container* creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.
- After the Servlet is instantiated successfully, the web container initializes the instantiated Servlet object. The container initializes the Servlet object by invoking the `init()` method

类先加载，实例再创建；`init` 初始化，`service` 管请求，`destroy` 最后销毁：类加载器将 Servlet 类加载到 Web 容器中。然后，Web 容器运行构造方法并创建 Servlet 实例。接着，容器调用 `init()` 初始化 Servlet。在生命周期的大部分时间里，Servlet 通过 `service()` 处理客户端请求，其中包括 `doGet()` 和 `doPost()` 。最后，调用 `destroy()` 终止 Servlet。

The `init()` Method

- The `init()` method is called only once after creating the servlet instance.
- not called for any user requests afterwards.
- Syntax of the `init()` method is given below:

```
public void init(ServletConfig config) throws ServletException {
    // Initialization code...
}
public void init() throws ServletException {
    // Initialization code...
}
```

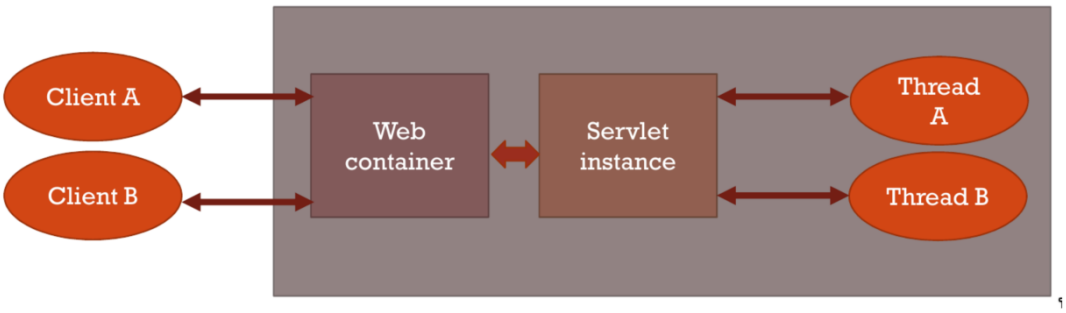
D

`service()` Method is Invoked

- The web container calls the `service()` method each time when request for the servlet is received.
- The `service()` method is the main method to perform the actual task.
- The web server calls the `service()` method to handle requests coming from the client(browsers) and to write the formatted response back to the client.

`service()` Method is Invoked

- Each time the server receives a request for a servlet, the server spawns a new thread and calls service. The `service()` method checks the HTTP request type (GET, POST, PUT, DELETE, etc.) and calls `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc. methods as appropriate.



`service()` Method

- The syntax of the service method of the Servlet interface is given below:

```
public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException {
    ....
}
```

you have nothing to do with `service()` method but you override either `doGet()` or `doPost()` depending on what type of request you receive from the client.

The `destroy()` Method

- A server calls the destroy method after all service calls have been completed.
- Java Servlet will be destroyed when container does not have enough memory and try to kill the idle objects include Java Servlet.
- Java Servlet will be destroyed when its idle time is over the preset timeout parameter.
- The `destroy()` method is called only once at the end of the life cycle of a servlet. This method gives your servlet a chance to close database connections, halt background threads, write cookie lists or hit counts to disk, and perform other such cleanup activities
- After the `destroy()` method is called, the servlet object is marked for garbage collection. The `destroy()` method definition looks like this .

```
public void destroy() {
// Finalization code...
}
```

3. Java Servlet Implementation: servlet Login example (passing the parameters)

核心：用户在 HTML 表单中输入数据，浏览器把数据作为 HTTP 参数发送给 Servlet；Servlet 使用 `request.getParameter()` 接收参数，然后生成响应。

文件	作用
`index.html`	欢迎页，提供**进入登录页面**的按钮
`loginPage.html`	包含用户**输入框**的 HTML 表单
`myLoginServlet.java`	接收参数、**处理**请求并生成响应
`web.xml`	**将** URL `/myLoginServlet` 映射到 Servlet 类

a. 从 `index.html` 进入登录页

Source Code: index.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Start Page</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    </head>
    <body>
        <h1>This is the welcome page</h1>
        <h2>Press the button to login</h2>
        <div>
            <form action="loginPage.html" method="get">
                <input type ="submit" value="Login Page">
            </form>
        </div>

    </body>
</html>
```

- <form>表示一个HTML表单；
- `action="loginPage.html"`， `action` 指定表单提交到哪里，这里代表点击按钮后，请求 `loginPage.html`
- `method="get"`， 浏览器使用 HTTP GET 请求访问该页面
- 所以点击按钮后，服务器返回 `loginPage.html`

b. 登录表单收集参数

Source Code: loginPage.html

```
<html>
    <head>
        <title>Login Page</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h1>This is the landing page</h1>
        <!--"action" indicates the Web component that
             receives the request when the form is submitted-->
        <form action="myLoginServlet" method="post">
            <label for='fname'>First Name:</label>
            <input type="text" id="fname" name="fname"><br><br>
            <label for='fname'>Last Name:</label>
            <input type="text" id="lname" name="lname"><br><br>
            <input type="submit" value="Submit">
        </form>
    </body>
</html>
```

- 表示提交表单时，请求 /myLoginServlet
- 但它不是直接写 Java 类名，而是请求一个 URL。这个 URL 稍后由 `web.xml` 映射到真正的 Servlet 类
- `id` 标识页面元素， `name` 标识提交给服务器的参数

c. Servlet 接收请求

Source Code: myLoginServlet.java

```java
package edu.mo.mpu.logindemo;

import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.util.Calendar;


public class myLoginServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
}
```

- `myLoginServlet` 继承 `HttpServlet`，因此可以处理 HTTP 请求

```
POST request
    ↓
HttpServlet.service()
    ↓
doPost()
    ↓
processRequest()
```

```
GET request
    ↓
service()
    ↓
doGet()
    ↓
processRequest()
```

- 两个方法都把处理工作交给 `processRequest()`，从而复用共同逻辑

d. `HttpServletRequest` 和 `HttpServletResponse`

Source Code: myLoginServlet.java

```java
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");

        // Receiving request Parameters
        String FirstName = request.getParameter("fname");
        String LastName = request.getParameter("lname");

        try (PrintWriter out = response.getWriter()) {
            /* TODO output your page here. You may use following sample code. */
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet myLoginServlet</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h3> Hello " + FirstName + " " + LastName + " welcome back!"+ "</h3>");
            out.println("<h3> How's it going?</h3>");
            out.println("<p>" + Calendar.getInstance().getTime() + "</p>");
            out.println("</body>");
            out.println("</html>");
        }
    }
```

`request`：读取客户端发来的内容 Read from request.

- 包含：表单参数、URL 参数、HTTP 方法、请求头、Cookie、Session 信息等

【重点：如何接收表单参数】

课件代码：

```java
String FirstName = request.getParameter("fname");
String LastName = request.getParameter("lname");
```

`request.getParameter(name)` 以字符串形式返回对应的请求参数值。

`response`：构造返回给客户端的内容 Write to response.

- `HttpServletResponse` 用于：设置响应类型、设置状态码、设置响应头、输出 HTML 或其他内容

设置内容类型：

```java
response.setContentType(
    "text/html;charset=UTF-8" //返回内容是 HTML，字符编码为 UTF-8
);
```

获得输出对象：

```java
PrintWriter out = response.getWriter();
```

PrintWriter 用于向 HTTP 响应体写文本。

```java
out.println("<!DOCTYPE html>");
out.println("<html>");
out.println("<head>");
out.println("<title>Servlet myLoginServlet</title>");
out.println("</head>");
out.println("<body>");
```

这些代码最终会在浏览器中组成一个 HTML 页面

怎样把参数放进响应?

```java
out.println(
    "<h3>Hello " + FirstName + " "
    + LastName + " welcome back!</h3>"
);
```

`try (PrintWriter ...)`

这是 Java 的 try-with-resources 语法， try-with-resources 会在代码块结束时自动关闭资源。

e. `web.xml` 如何找到 Servlet?

Source Code: web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="5.0" xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
        https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd">

    <servlet>
        <servlet-name>myLoginServlet</servlet-name>
        <servlet-class>edu.mo.mpu.logindemo.myLoginServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>myLoginServlet</servlet-name>
        <url-pattern>/myLoginServlet</url-pattern>
    </servlet-mapping>

    <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
</web-app>
```

- 前半部分声明：名为 `myLoginServlet` 的 Servlet，对应哪个 Java 类
- 后半部分声明：URL `/myLoginServlet` 由名为 `myLoginServlet` 的 Servlet 处理

项目	示例	含义
URL pattern	`/myLoginServlet`	浏览器请求路径
Servlet name	`myLoginServlet`	配置中的逻辑名称
Servlet class	`edu...myLoginServlet`	真正的 Java 类

> **总结**：在登录示例中，HTML 表单收集用户的名字和姓氏。输入框的 `name` 属性 `fname` 和 `lname` 会成为 HTTP 请求参数的名称。用户点击 Submit 后，浏览器向 `/myLoginServlet` 发送 POST 请求。该 URL 通过 `web.xml` 或 `@WebServlet` 注解映射到 Servlet。Web 容器随后调用 Servlet 的 `doPost()` 方法。Servlet 使用 `request.getParameter("fname")` 和 `request.getParameter("lname")` 取得提交值，再设置响应类型、获取 `PrintWriter`，最后生成包含这些参数的 HTML 响应。

4. doGet() ; doPost ;Deployment Descriptor (web.xml); @WebServlet

- `doGet()` 、 `doPost()` ： 请求到达 Servlet 后，如何处理
- `web.xml` 、 `@WebServlet` ： 请求如何找到 Servlet

```
浏览器请求 /hello
      ↓
web.xml 或 @WebServlet 完成 URL 映射
      ↓
找到 HelloWorldServlet
      ↓
GET  → doGet()
POST → doPost()
      ↓
Servlet 生成响应
```

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
  throws ServletException, IOException {
}


protected void doPost(HttpServletRequest req, HttpServletResponse resp)
  throws ServletException, IOException { ... }
...
}
```

`doGet()` 用来处理 HTTP GET 请求，通常用于获取或查询数据

例如访问：

```
http://localhost:8080/MyApplication/hello?name=Tom
```

浏览器发送的请求类似：

```
GET /MyApplication/hello?name=Tom
```

Servlet：

```java
@Override
protected void doGet(
    HttpServletRequest request,
    HttpServletResponse response)
throws IOException {

    String name = request.getParameter("name");

    response.setContentType(
        "text/html;charset=UTF-8"
    );

    response.getWriter().println(
        "<h1>Hello " + name + "</h1>"
    );
}
```

执行结果：

```
name = "Tom"
```

浏览器显示：

```
Hello Tom
```

`doPost()`：处理 POST 请求

方法格式：

```java
@Override
protected void doPost(
    HttpServletRequest req,
    HttpServletResponse resp)
throws ServletException, IOException {
    // Handle POST request
}
```

`doPost()` 用来处理 HTTP POST 请求，通常用于提交表单、创建数据或执行**会改变服务器状态的操作**。：

```html
<form action="hello" method="post">
  <input type="text" name="name">
  <input type="submit" value="Submit">
</form>
```

Servlet：

```java
@Override
protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response)
throws IOException {

    request.setCharacterEncoding("UTF-8");

    String name = request.getParameter("name");

    response.setContentType(
        "text/html;charset=UTF-8"
    );

    response.getWriter().println(
        "<h1>Welcome, " + name + "</h1>"
    );
}
```

GET 的特点

- 通常用于查询、读取数据
- 参数通常附在 URL 后面
- URL 可以收藏、复制和缓存
- 不适合传密码等敏感内容
- 按语义不应修改服务器数据

POST 的特点

- 通常用于提交或创建数据
- 参数通常放在 HTTP request body 中
- 不适合收藏为书签
- 可能改变服务器状态
- 传密码时仍然必须使用 HTTPS；POST 本身不加密

Deployment Descriptor：`web.xml`

Deployment Descriptor（DD）= 部署描述符， 它负责描述 Web 应用如何部署和运行

`web.xml` 通常位于：

```
WEB-INF/web.xml
```

Mapping the Servlet: *web.xml*

- Using *web.xml*

```
<servlet>
    <servlet-name>HelloWorld</servlet-name>
    <servlet-class>demo.HelloWorldServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name></servlet-name>
    <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

- http://localhost:8080/MyApplication/hello
- What is an URL pattern?
- Why do we need a servlet name?

`web.xml` 三个字段分别是什么?

1. `<servlet-class>`： 表示真正处理请求的 Java 类，必须写完整类名（包名 + 类名）
2. `<servlet-name>`： 配置文件内部使用的逻辑名称
3. `<url-pattern>`： 表示用户请求哪个路径时，应由这个 Servlet 处理

`@WebServlet` 注解

Mapping the Servlet: `@WebServlet`

Create a mapping: *class - name - url pattern*

- Using `@WebServlet` annotation

```
    @WebSersvlet (name = "HelloWorld", urlPatterns = {"/hello"})
    public class HelloWorldServlet extends HttpServlet {
        ......
    }
```

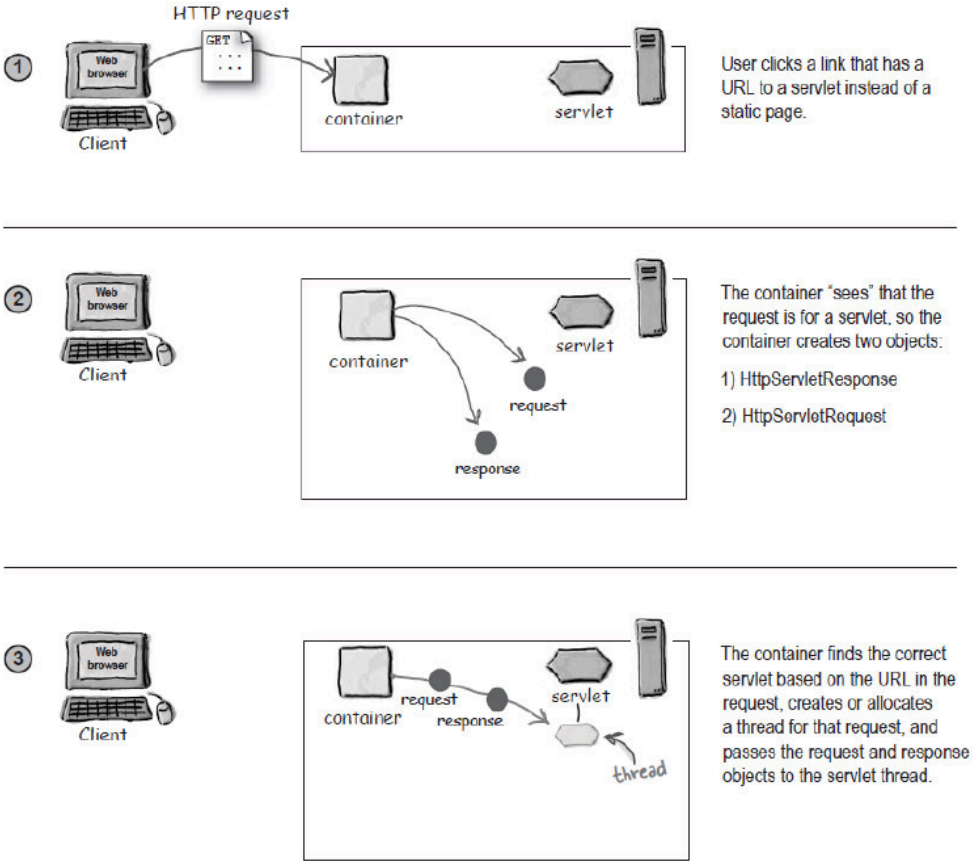
`@WebServlet` 是另一种配置 Servlet 映射的方法，可以直接写在 Java 类上，代替 `web.xml` 中对应的 Servlet 声明与映射。

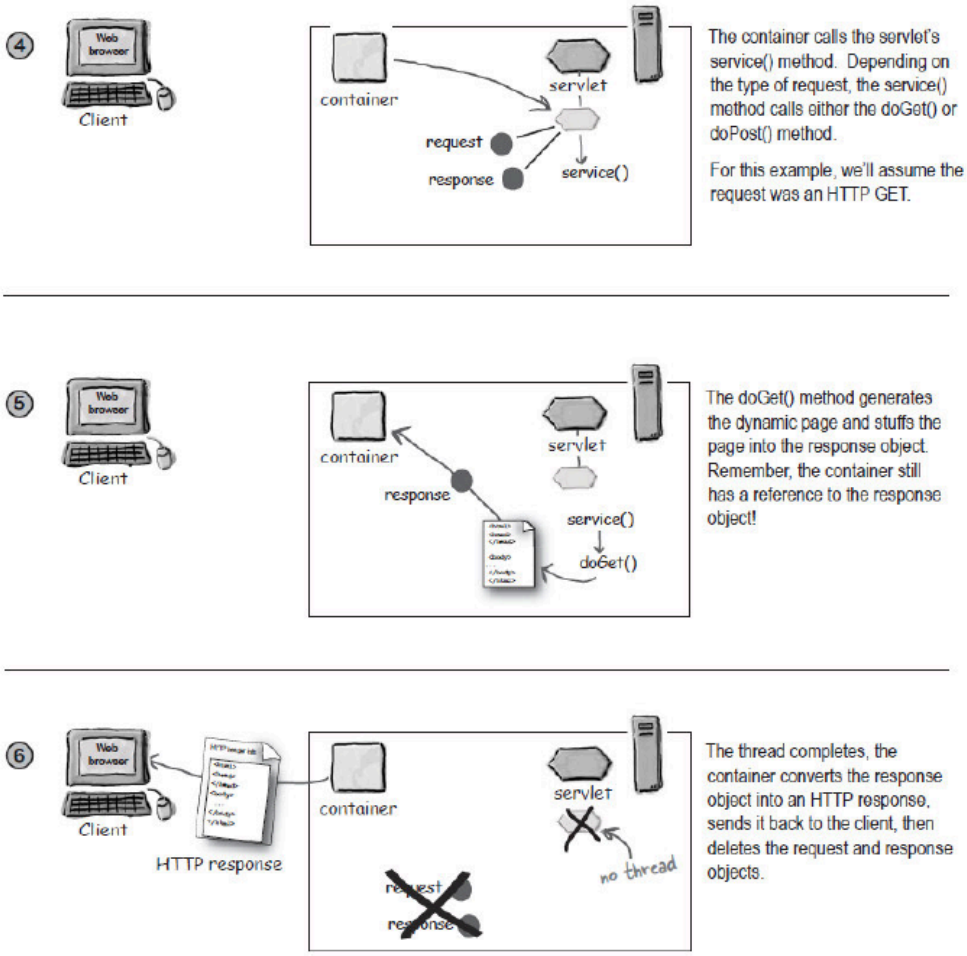
部分	含义
`name="HelloWorld"`	Servlet 的逻辑名称
`urlPatterns={"/hello"}`	Servlet 对应的请求路径
`HelloWorldServlet`	真正的 Java 类

`web.xml` 在外面配，`@WebServlet` 在类上配。

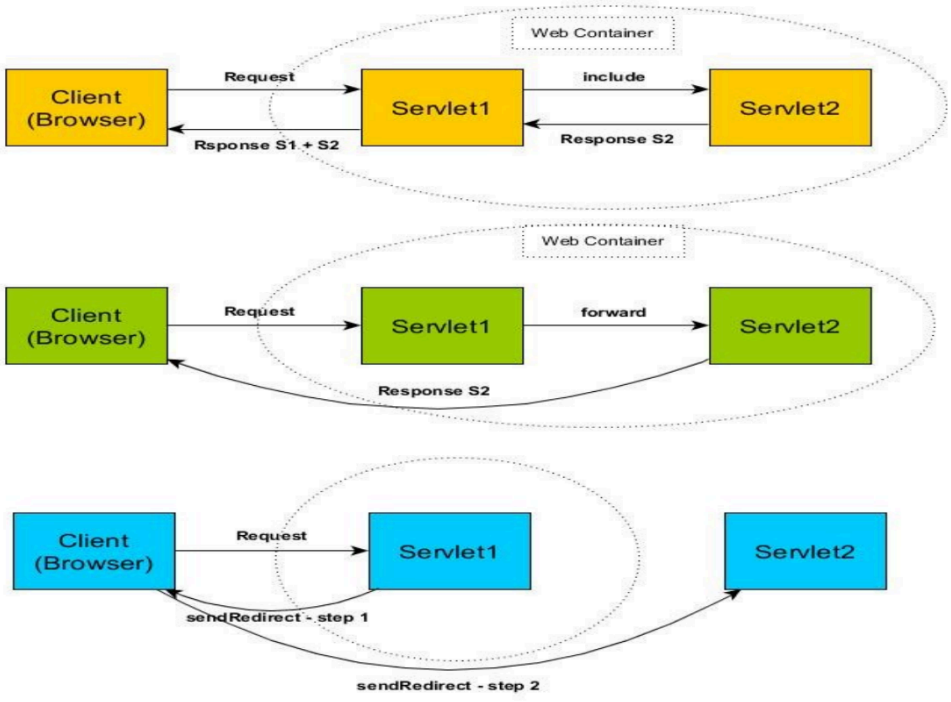
> `doGet()` 处理 HTTP GET 请求，通常用于获取数据；`doPost()` 处理 HTTP POST 请求，通常用于提交或创建数据。Web 容器根据 HTTP 方法把请求分派给相应的方法。部署描述符 `WEB-INF/web.xml` 可以声明 Servlet 类，并通过逻辑 Servlet 名称将 URL 路径映射到该类；也可以使用 `@WebServlet` 注解，直接在 Java 类上声明 Servlet 及其 URL 映射。

5. Simple transaction flow





6. Servlet communication (forward, include, redirect)



7. Session management over http

Web Session

- Web Session = the “dialogue” between an user and a web application
  - login → buy stuff → logout
- Session Data = information stored on the server specific to a certain “dialogue”
  - user info, shopping cart,etc.
- Over HTTP, sessions are **maintained by the application server** (on demand) using a:
  - session ID, usually an UUID
    `jsessionid=1A530637289A03B07199A44E8D531427`

1. **Web session**： 是用户与 Web 应用之间的一段“对话”或连续交互

例如，login –> buy stuff –> logout，这几个操作不是彼此无关的请求，而是同一个用户从登录到退出的完整使用过程，属于同一个session。

2. **Session data**： 是保存在服务器上、专属于某一次会话的信息

例如： `user info` ：用户信息， `shopping cart` ：购物车内容

Session 是交互过程，Session Data 是这段过程中需要记住的信息。

3. **Session ID**：应用服务器使用 Session ID 来维持和识别 Session， Session ID 通常是一个 UUID，也就是一个具有唯一性的编号，告诉服务器当前请求属于哪一个 Session，服务器通过 Session ID 将请求与正确的 Session 对应起来

多个用户会同时访问服务器。服务器需要判断：

- 当前请求来自哪个会话
- 应该使用哪一份用户信息
- 应该读取哪一个购物车

服务器如何维持 Session?

Session Management over HTTP

A server maintains a session using either:

- **Cookies (`jsessionid`)**
  - first HTTP request: the server creates a **jsessionid cookie** and sends it to the server (inside the response)
  - subsequent requests: the cookie is included inside the client requests – the server reads the id and identifies the session
- URL rewriting
  - `https://localhost:8080/myapp/somerequest.html;jsessionid=1A530637289A03B07199A44E8D531427`

方法一：Cookies（`jsessionid`）

第一次 HTTP 请求，用户第一次访问时，服务器：

1. 创建一个 Session
2. 生成 `jsessionid`
3. 将包含 `jsessionid` 的 Cookie 放在 HTTP Response 中
4. 发送给客户端

后续 HTTP 请求，客户端在后续请求中带上这个 Cookie， 服务器读取这个 ID，并识别对应的会话

方法二：URL Rewriting

如果使用 URL rewriting，Session ID 会被加入 URL：

Plain Text

```
https://localhost:8080/myapp/somerequest.html; //原来的请求地址
jsessionid=1A530637289A03B07199A44E8D531427 //附加在地址中的 Session ID
```

客户端访问这个 URL 时，服务器可以从 URL 中读取 `jsessionid`，从而识别 Session。

方法	Session ID 放在哪里
Cookie	Cookie 中
URL rewriting	URL 中

8. Java Bean

- EAR (Enterprise ARchive)
  - A standard JAR file used by Java EE for packing the whole application (program and configuration) into a single archive for deploy to different platform.
  - It must contain a DD (Deployment Descriptors) XML to describe how the application will be deployed.
  - It contains a WAR file (web application) for deployment
  - It may contain Enterprise Java Beans (Message Beans and Session Beans) for remote access.

JavaBean 是一种遵循特定编码约定的普通 Java 类，主要用于封装数据，Reusable Java class that encapsulates data through properties, constructors, getters, and setters. 通过属性、构造方法、getter 和 setter 封装数据。